Southern New Hampshire University

CS 505 Module Five Activity: Graph Theory and Graphical Representations Template

Complete this template by replacing the bracketed text with the relevant information.

Part One: Prim's and Kruskal's Algorithms

For Part One, implement and analyze Prim's and Kruskal's algorithms, exploring their performances and applications.

1. Implement both Prim's and Kruskal's algorithms using pseudocode.

   Answer:

```
Prim(G):
        MST = Empty Set
        visited = Empty Set
        choose starting vertex as s
        add s to visited
        while visited does not contain all vertices:
                minEdge = smallest edge (u, v, w)
                        where:
                                u is visited
                                v is unvisited
                                w is weight/edge length
                add (u, v, w) to MST
                add v to visited
        return MST

Kruskal(G):
        MST = Empty Set
        sort edges (u, v, w) by increasing weight (w)
        for each edge (u, v, w):
                if adding edge does not create a cycle:
                        add (u, v, w) to MST
                if MST contains n-1 edges:
                        stop
        return MST
```

2. Test the implementations on various graph types, including dense and sparse graphs, weighted and unweighted.

Answer:

To test Prim's and Kruskal's algorithms, I would run both algorithms on several types of graphs and compare the results. Since both algorithms should produce a minimum spanning tree, the final MST should connect all vertices, contain no cycles, and have exactly n - 1 edges, where n is the number of vertices.

Dense Graph:
A dense graph has many edges. I would test both algorithms on a graph in which most vertices are connected to several others. This checks whether the algorithms can still choose the lowest-weight edges without creating cycles.

Dense Graph Example:
Vertices: A, B, C, D
Edges:
A-B (1), A-C (2), A-D (4),
B-C (2), B-D (3),
C-D (1)

Both Prim's and Kruskal's algorithms produced an MST with total weight 4.

Sparse Graph:
A sparse graph has fewer edges. I would test both algorithms on a graph where vertices have only a few connections. This checks whether the algorithms can still form a spanning tree when fewer edge choices are available.

Sparse Graph Example:
Vertices: A, B, C, D
Edges:
A-B (1), B-C (2), C-D (3)

Both algorithms produced the same MST because there was only one possible spanning tree.

Weighted Graph:
A weighted graph has numerical values assigned to edges. This is the main type of graph used for Prim's and Kruskal's algorithms. I would verify that both algorithms choose edges that create the lowest total weight.

Weighted Graph Example:
Vertices: A, B, C
Edges:
A-B (2), A-C (5), B-C (1)

Both Prim's and Kruskal's algorithms selected edges B-C and A-B, producing an MST with a total weight of 3.

Unweighted Graph:
An unweighted graph has no edge weights. Since MST algorithms require edge weights, I would treat each edge as having weight 1. In this case, any spanning tree with n - 1 edges would be acceptable because all edges have equal cost.

Unweighted Graph Example:
Vertices: A, B, C, D
Edges:
A-B, B-C, C-D, A-D

Since all edges can be treated as having equal weight, multiple valid spanning trees are possible. Both algorithms still produced spanning trees with n - 1 edges and no cycles.

For each test, I would check that:
1. All vertices are connected.
2. The MST contains no cycles.
3. The MST has exactly n - 1 edges.
4. The total weight is as small as possible.
5. Prim's and Kruskal's algorithms produce the same total MST weight, even if they choose different edges when equal weights exist.

3. Analyze the time and space complexity of both algorithms.
   Answer:
   Prim's algorithm has a time complexity of $O(V^2)$ in its basic implementation, where V represents the number of vertices. This is because the algorithm repeatedly searches for the smallest edge connecting the current tree to an unvisited vertex. Its space complexity is O(V) because it stores the visited vertices and the edges included in the minimum spanning tree.

   Kruskal's algorithm has a time complexity of O(E log E), where E represents the number of edges. This is because the algorithm must sort all edges by weight before constructing the minimum spanning tree. Its space complexity is O(V + E) because it stores the graph edges, the minimum spanning tree, and the information needed to track connected components and avoid cycles.

4. Compare the performance of Prim's and Kruskal's algorithms.
   Answer:
   Prim's and Kruskal's algorithms both produce a minimum spanning tree, but their performance differs depending on the structure of the graph.

   Prim's algorithm has a time complexity of $O(V^2)$ in its basic implementation, where V represents the number of vertices. Because Prim's algorithm grows a single tree by repeatedly selecting the smallest connecting edge, it often performs well on dense graphs where many edges exist between vertices.

   Kruskal's algorithm has a time complexity of O(E log E), where E represents the number of edges. This is because the algorithm must first sort all edges by weight before constructing the minimum spanning tree. Kruskal's algorithm often performs better on sparse graphs because there are fewer edges to sort and process.

   Regarding space complexity, Prim's algorithm generally requires less memory, with a space complexity of O(V), since it primarily stores visited vertices and the edges in the minimum spanning tree. Kruskal's algorithm requires O(V + E) space because it stores all graph edges and uses additional structures to track connected components and prevent cycles.

5. Describe how MST algorithms can be applied to real-world problems.
   Answer:
   MST algorithms can be applied to many different real-world problems.

   One of these is a delivery network, such as Amazon or UPS. They need to calculate the minimum cost of transporting products between warehouses, distribution centers, and delivery points.

   Another way that I immediately thought of when reading about this is in map design for TTRPGs, which could even apply to road building and structures. MST algorithms can be used to find the shortest trade routes in fantasy settings or the cheapest, most efficient way to build roads between cities, towns, etc.

6. Explain the concept of minimum spanning forests for disconnected graphs.

Answer:

A minimum spanning forest (MSF) for a disconnected graph is a collection of minimum spanning trees (MSTs). When a graph contains disconnected components, it is considered disconnected. Because of this, a single minimum spanning tree cannot connect all vertices, resulting in multiple MSTs that together form a minimum spanning forest.

When using algorithms to find an MSF, Prim's algorithm must be run separately for each connected component because it builds the tree by expanding from a starting vertex. In contrast, Kruskal's algorithm naturally creates a minimum spanning forest because it selects the shortest edges without requiring the graph to be fully connected. As a result, it forms separate minimum spanning trees for each disconnected component, collectively creating the forest.

7. Implement Kruskal's algorithm using a disjoint-set data structure with path compression.

Answer:

```
class DisjointSet:
    def __init__(self, vertices):
        self.parent = {v: v for v in vertices}
        self.rank = {v: 0 for v in vertices}

    def find(self, vertex):
        if self.parent[vertex] != vertex:
            self.parent[vertex] = self.find(self.parent[vertex])
        return self.parent[vertex]

    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)

        if root_u == root_v:
            return False

        if self.rank[root_u] < self.rank[root_v]:
            self.parent[root_u] = root_v
        elif self.rank[root_u] > self.rank[root_v]:
            self.parent[root_v] = root_u
        else:
            self.parent[root_v] = root_u
            self.rank[root_u] += 1

        return True


def kruskal(vertices, edges):
    mst = []
    disjoint_set = DisjointSet(vertices)
```

```
    edges.sort(key=lambda edge: edge[2])

    for u, v, weight in edges:
        if disjoint_set.union(u, v):
            mst.append((u, v, weight))

    return mst


vertices = ["A", "B", "C", "D"]

edges = [
    ("A", "B", 1),
    ("A", "C", 4),
    ("B", "C", 2),
    ("B", "D", 5),
    ("C", "D", 3)
]

mst = kruskal(vertices, edges)

print("Minimum Spanning Tree:")
for edge in mst:
    print(edge)
```

Part Two: Ford-Fulkerson Algorithm

For Part Two, implement and analyze Ford-Fulkerson Algorithm, exploring its performance and applications.

1. Implement the Ford-Fulkerson algorithm using pseudocode.

   Answer:

```
FordFulkerson(Graph, source, sink):

    maxFlow = 0

    for each edge (u, v) in Graph:
        flow(u, v) = 0

    while augmenting path exists from source to sink:
        path = findPath(Graph, source, sink)
        pathFlow = minimum residual capacity in path

        for each edge (u, v) in path:
            flow(u, v) = flow(u, v) + pathFlow
            flow(v, u) = flow(v, u) - pathFlow

        maxFlow = maxFlow + pathFlow
    return maxFlow
```

2. Test the implementation on various flow network examples, including simple and complex networks.

   Answer:

   The Ford-Fulkerson implementation can be tested using both simple and complex flow networks. A simple test could include one source, one sink, and only one or two paths between them. This would confirm that the algorithm can find an augmenting path, calculate the minimum residual capacity, and update the total flow correctly.

   Simple Network Example:
   Source → A (10)
   Source → B (5)
   A → Sink (10)
   B → Sink (5)

   The Ford-Fulkerson algorithm found a maximum flow of 15.

   A more complex test could include several intermediate vertices, multiple possible paths from the source to the sink, and edges with different capacities. This would test whether the algorithm can continue to find augmenting paths until no valid paths remain. For each test, the final maximum flow should be checked to ensure it does not exceed any edge capacity and that the total flow entering the sink equals the total flow leaving the source.

   Complex Network Example:
   A network with multiple intermediate vertices and overlapping paths was tested to ensure the algorithm could repeatedly update residual capacities and continue finding augmenting paths until no valid paths remained.

3. Analyze the time and space complexity of the Ford-Fulkerson algorithm.

   Answer:

   Time:
   The time complexity of the Ford-Fulkerson algorithm depends on the number of augmenting paths that must be found before reaching the maximum flow. The time complexity is typically written as: $O(E \times F)$ where E is the number of edges in the graph and F is the maximum flow of the network.

   Each search for a path can take time, and the algorithm may need to repeat multiple times until no valid paths remain.

   Space:
   The space complexity is $O(V + E)$ because the algorithm must store the graph, residual graph, flow values, and edge capacities. In this case, V is the number of vertices, while E is the number of edges.

4. Describe the overall performance of the Ford-Fulkerson algorithm.

   Answer:

   Overall, the Ford-Fulkerson algorithm performs well for finding the maximum flow in a network, especially in smaller or moderately sized graphs. The algorithm works by repeatedly finding

augmenting paths with available capacity and pushing additional flow through them until no valid paths remain.

The overall performance depends heavily on the path selection. If inefficient paths are repeatedly chosen, the algorithm may require many more iterations to reach the maximum flow. As the algorithm continues, the residual capacities on the edges change, affecting which paths are still available. Eventually, the remaining capacities become too small or completely full, preventing any additional augmenting paths from being found.

Ford-Fulkerson is effective because it continuously updates the residual graph and adjusts the flow after each iteration. However, for larger or more complex networks, optimized versions such as the Edmonds-Karp algorithm are often preferred because they select paths more efficiently and provide more consistent performance (Kogler, 2026).

Kogler, J. [Jakob Kogler]. (2026, January 21). Maximum flow - Ford-Fulkerson and Edmonds-Karp [Video]. YouTube. https://www.youtube.com/watch?v=Tl90tNtKvxs

5. Describe how network flow algorithms can be applied to real-world problems.
   Answer:
   Network flow algorithms can be applied to many real-world problems involving the movement of resources through a system. One example is **traffic management**, where roads act as edges and intersections act as vertices. The algorithm can help determine the maximum amount of traffic that can move through the road network without causing congestion.

   Another example is **computer networking**, where network flow algorithms are used to determine how much data can be transferred between servers over a network while avoiding bandwidth bottlenecks. Companies that manage large amounts of online traffic rely on these algorithms to optimize data transfer and reduce delays.

   Network flow algorithms can also be used in **supply chain management** to determine the maximum amount of goods that can move among warehouses, factories, and stores while accounting for transportation constraints and capacities.

6. Explain the concept of min-cut max-flow theorem.
   Answer:
   The min-cut max-flow theorem states that the maximum amount of flow that can move from the source to the sink in a network is equal to the capacity of the smallest cut in the graph.

   A cut is a separation of the graph into two groups, with the source on one side and the sink on the other. The capacity of the cut is found by adding the capacities of the edges crossing between the two groups.

   Essentially, the theorem shows that the maximum flow through a network is limited by its biggest bottleneck. Once the smallest cut becomes full, no additional flow can move through the network.